



IIC2343 - Arquitectura de Computadores (II/2025)

Ayudantía 15

Ayudantes: Daniela Ríos (danielaarp@uc.cl), Alberto Maturana (alberto.maturana@uc.cl), Tomás López Massaro (tomas.lopezm20@uc.cl)

Pregunta 1: Preguntas Conceptuales

- (a) Describa cómo se puede modificar el computador básico con *pipeline* para que las predicciones de salto erróneas pierdan dos ciclos en vez de tres.

Solución: Como la *flag Z* se computa en la etapa **EX**, una posible modificación consiste en posicionar la *Jump Unit* en dicha etapa para poder determinar en una iteración menos si corresponde realizar el salto o no. De esta forma, tanto para saltos incondicionales como condicionales se requerirá un *flush* solo para las etapas **IF** e **ID** si corresponde, perdiendo dos ciclos y no tres.

- (b) Considere el siguiente código escrito para el assembly del computador básico con *pipeline*:

```
ADD A,B  
MOV (var0), A  
MOV (var1), B
```

1. Indique con un diagrama los *hazards* que se generan al ejecutar el código anterior.

Solución: Hay un hazard entre las líneas 1 y 2 ya que el valor de **A** no ha alcanzado a hacer **WB** cuando se requiere su valor para la instrucción siguiente.

2. Indique que sucede al intercambiar el orden de la segunda y tercera línea del código. ¿Cambia el resultado final? En caso de existir *hazards*, indíquelos con un diagrama.

Solución: Hay un hazard entre las líneas 1 y 3, ya que se requiere que el valor del registro **A** que aún no ha hecho **WB** cuando pasa la línea 3 por **EX**

3. Para los dos casos anteriores, en caso de existir *hazards*, indique cómo los detectan las *forwarding units* correspondientes, especificando las señales de control participantes.

Solución: En el primer caso, la FU1 considera las señales ID/EX.ALU_A, MEM/WB.Load_B que indican que se requiere el valor de A antes que haya hecho WB. Para el segundo caso, la FU2 deberá leer MEM/WB.Load_A, EX/MEM.Mem_In y MEM/WB.Reg_In. El primer par de señales identifica al *hazard* y la tercera indica que la FU2 puede subir la señal Forward_Din para hacer el *forward* correspondiente.

(c) Estime, en términos de N , el número de ciclos que toma la ejecución de un programa de N instrucciones en el computador básico con *pipeline* si en este:

- El 10 % de las instrucciones son de salto y, en el 25 % de estas, el salto se realiza.
- El 30 % de las instrucciones son de escritura de memoria y, el 40 % de estas, escribe datos obtenidos en la instrucción anterior o previa a la anterior.
- El 20 % de las instrucciones son de lectura de memoria y, en el 70 % de estas, se usa el dato leído en la siguiente instrucción.

Solución: Si se tiene un programa de N instrucciones, en el **mejor caso** este se ejecutará en $N + 4$ ciclos. No obstante, para obtener una estimación correcta es necesario, además, incluir los ciclos adicionales por *stalling* y los perdidos por *flushing*:

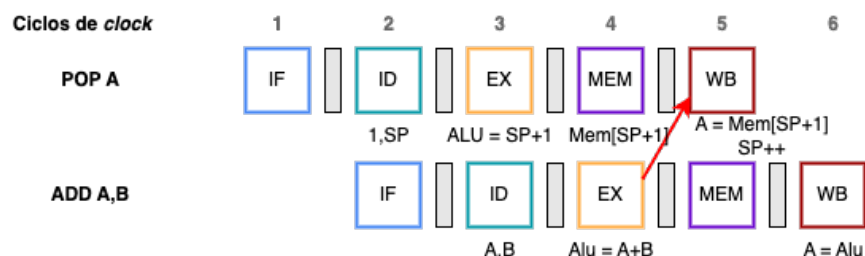
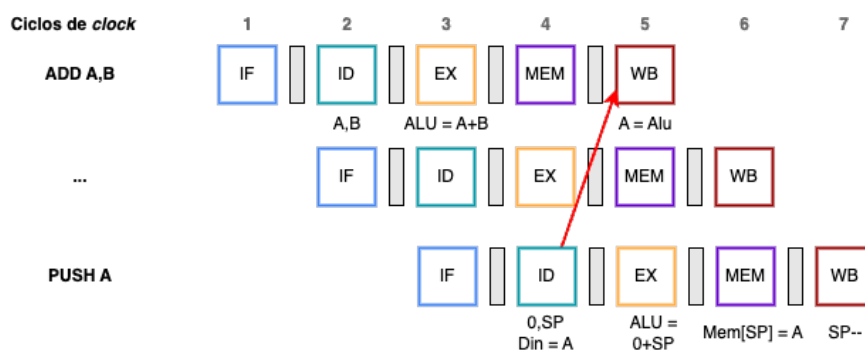
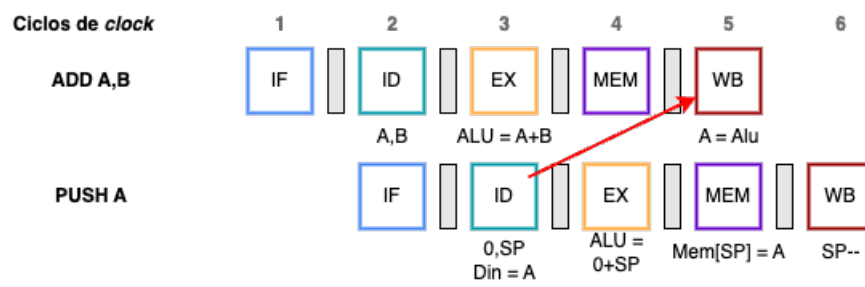
- Si en el 25 % de las instrucciones de salto estos se llevan a cabo, entonces la predicción realizada en el *pipeline* es errónea en dicho porcentaje. Este error genera una pérdida de 3 ciclos, por lo que proporcionalmente se pierden $3 \times 0,1 \times 0,25 \times N = 0,075N$ ciclos.
- Si en el 70 % de las instrucciones de lectura se usa inmediatamente el dato leído, en dichos casos necesitaremos hacer *stalling* por un ciclo (ya sea por burbuja o por NOP). Esto implica un aumento de ciclos igual a $1 \times 0,2 \times 0,7 \times N = 0,14N$.
- Las escrituras de memoria per se no generan *stalling* ni *flushing*, solo *forwarding*. Se pueden obviar para el cálculo.

De esta forma, el número de ciclos estimado para que ejecute este programa es igual a $N + 0,14N + 0,075N + 4 = 1,215N + 4$.

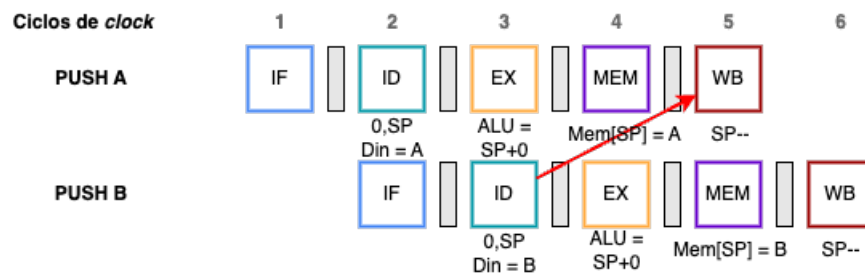
Pregunta 2: Paralelismo a nivel de instrucción (ILP)

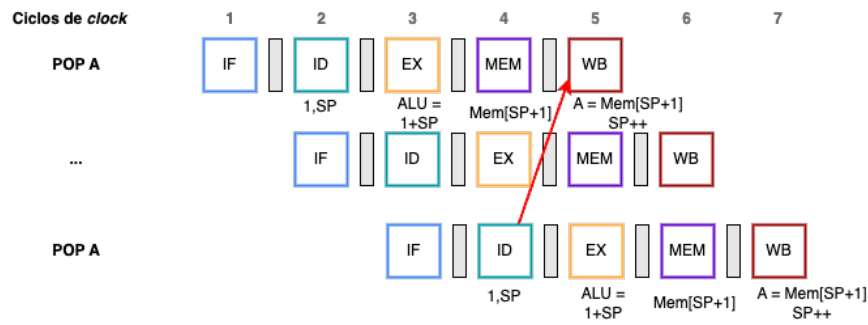
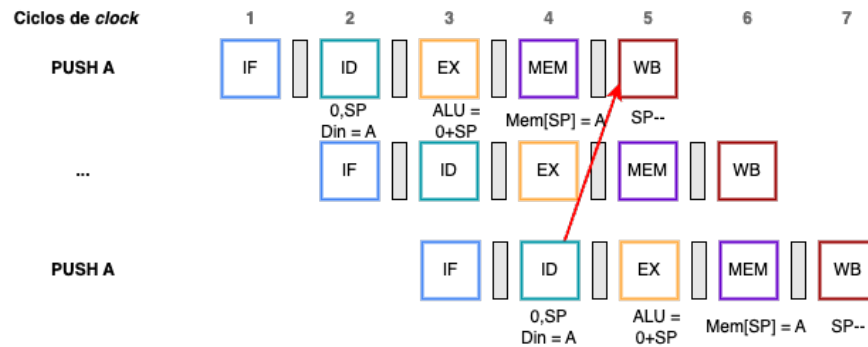
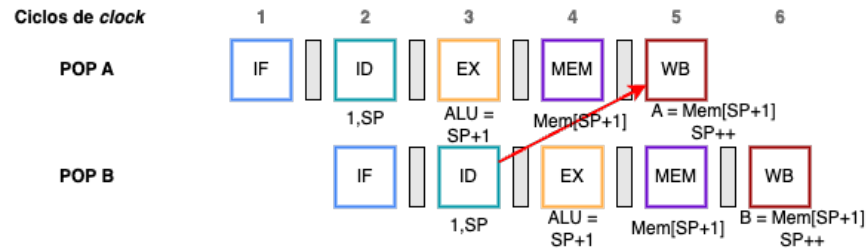
Suponga que se modifica el computador básico con *pipeline* para reincorporar las instrucciones de manejo de *stack*: PUSH A/B y POP A/B. Indique, con ejemplos, los tipos de *hazard* que podrían surgir de las instrucciones añadidas en casos distintos, explicando en cada ejemplo la etapa en la que surge la dependencia. Se considerarán “casos distintos” si se detectan con combinaciones de señales distintas.

Solución: Como PUSH A y PUSH B corresponden a instrucciones de escritura de memoria, mientras que POP A y POP B son de lectura de memoria, entonces pueden generar los mismos tipos de *hazards* de estos tipos de instrucción. A continuación, se muestran tres casos distintos:

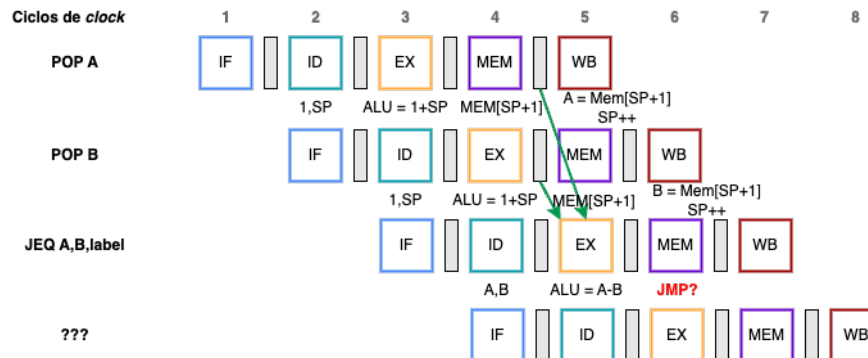


Por otra parte, si asumiéramos que el *stack pointer* se actualiza en la etapa **Writeback**, podrían surgir otros cuatro casos distintos:





Por último, los *hazards* de control ocurren cuando existe una instrucción POP antes de un salto condicional. Se muestra a continuación con un solo ejemplo todos los casos existentes:



Pregunta 3: Paralelismo a nivel de instrucción (ILP) [P3-I3-2024-1]

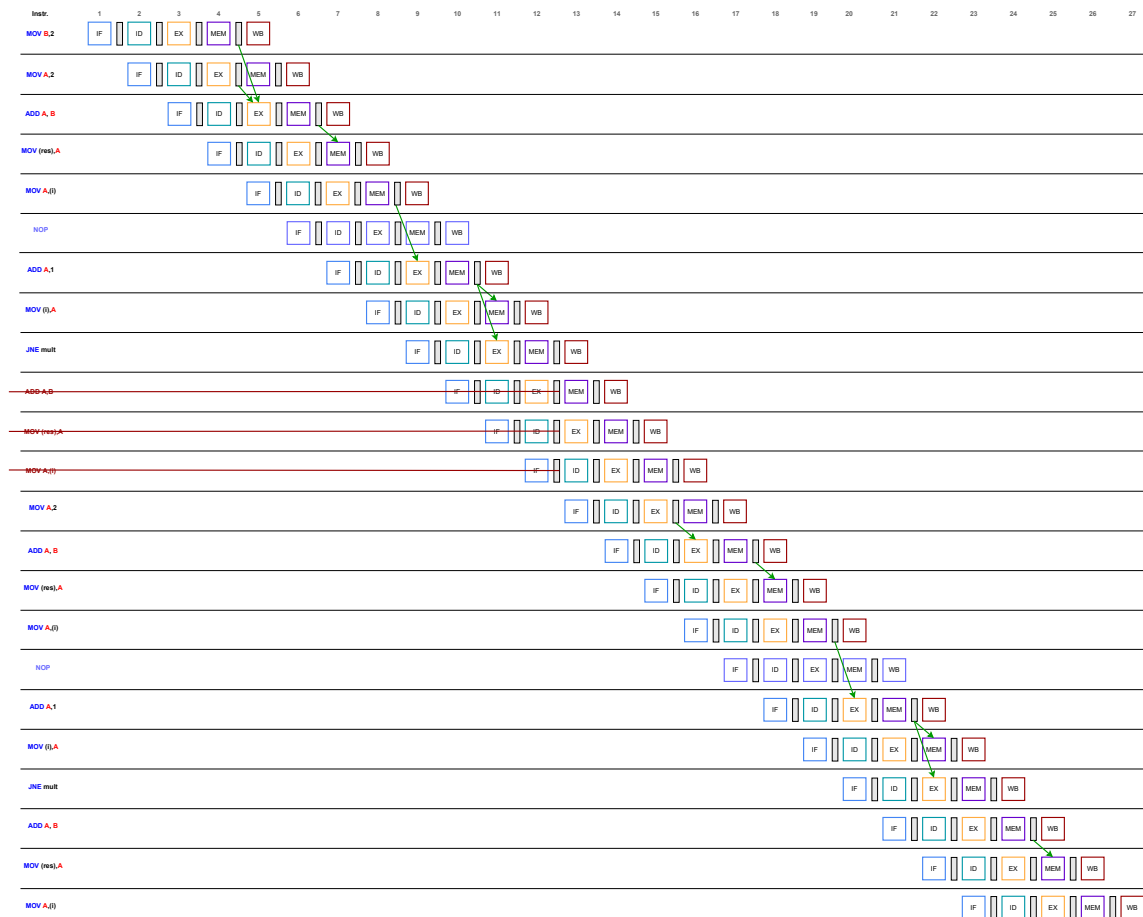
A partir del siguiente programa del computador básico con *pipeline*:

```
DATA:
    res 0
    i   0
CODE:
    MOV B,2
    mult:
        MOV A,2
        ADD A,B
        MOV (res),A
        MOV A,(i)
        ADD A,1
        MOV (i),A
        JNE mult
        ADD A,B
        MOV (res),A
        MOV A,(i)
```

Determine el número de ciclos que demora su ejecución detallando en un diagrama los estados del *pipeline* por instrucción. Asuma que el manejo de *stalling* es por *software* a través de la instrucción NOP y que la unidad predictora de saltos asume que estos nunca se realizan, independiente de que estos sean condicionales o incondicionales. Indique en el diagrama adjunto al enunciado cuando ocurre *forwarding* (con flechas desde el registro hacia la etapa que corresponda), *stalling* y *flushing* (con tachado en las instrucciones *flushheadas*).

The diagram illustrates the execution of a program with 32 instructions, organized into 16 horizontal tracks. Each track represents an instruction, and the flow of execution is indicated by arrows connecting the operations across the tracks. The operations are color-coded: P (blue), IO (light blue), EX (orange), MEM (purple), and WB (red). Data values are represented by numbers in boxes, ranging from 0 to 31. The flowchart shows a sequential execution of instructions, with each instruction track containing a unique sequence of operations and data values. The diagram is a detailed representation of the program's execution, showing the sequence of operations and the data values used in each instruction.

Solución: A continuación, se muestra el *pipeline* resultante de la ejecución del programa.



Del *pipeline* anterior se deduce que el programa termina de correr después de 27 ciclos. Se otorga puntaje de la siguiente forma: **0.3 ptos.** por cada *stalling* correctamente detectado (**0.6 ptos.** por todas las detecciones); **0.4 ptos.** por la detección correcta del *flushing*; **0.15 ptos.** por cada *forwarding* correctamente detectado (**1.8 ptos.** por todas las detecciones); y **0.2 ptos.** por la deducción correcta de la cantidad de ciclos en la que corre el programa. No hay descuento si se incluyen las instrucciones de escritura de memoria correspondientes al segmento DATA, pero solo si la ejecución de estas se describe de forma correcta.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

